
Lab 2 · Tema 2

Calidad y preparación de datos telco

Caso transversal: operador de telecomunicaciones

Entorno: AWS Academy Learner Lab

Servicios: Amazon S3 · AWS Glue Data Catalog · Amazon Athena · AWS Glue DataBrew
(opcional)

Duración estimada: 150-180 minutos · Nivel: inicial-intermedio

1. Idea general del laboratorio

Este laboratorio corresponde al Tema 2 y continúa directamente el trabajo del Lab 1. En aquella primera práctica cargamos en Amazon S3 las seis trazas principales de la operadora y las dejamos consultables desde Athena. Sin embargo, hicimos algo que en la práctica real nunca debe hacerse: confiar en que los datos estaban limpios. No lo estaban, y rara vez lo están.

Toda traza de un sistema complejo real llega con problemas de calidad: valores ausentes, duplicados, formatos inconsistentes, categorías escritas de mil maneras, valores imposibles, claves que no casan entre tablas y sesgos de captura. Si construimos modelos de IA sobre datos sucios, el resultado será un modelo que aprende del ruido y de los errores tanto como de la señal. La célebre máxima "garbage in, garbage out" no es un eslogan: es la causa más frecuente de fracaso en proyectos de analítica.

El objetivo de este laboratorio es que el alumno aprenda a perfilar, limpiar e integrar los datos de la operadora antes de aplicar cualquier técnica de inteligencia artificial. Trabajaremos con versiones deliberadamente "sucias" de los datasets del caso telco, descubriremos sus problemas con consultas de perfilado en Athena, los corregiremos mediante transformaciones SQL y produciremos una capa de datos preparada y fiable. Opcionalmente, exploraremos AWS Glue DataBrew para realizar parte de este trabajo sin escribir código.

Objetivos didácticos

Al finalizar el lab, el alumno debe ser capaz de:

1. Realizar un perfilado de datos: contar nulos, duplicados, valores distintos y rangos de cada columna.
2. Detectar problemas de calidad típicos: ausencias, duplicados, inconsistencias de formato, valores imposibles y claves huérfanas.
3. Estandarizar categorías y formatos (regiones, tipos de contrato, estados) mediante funciones SQL.
4. Tratar valores nulos y valores imposibles con criterios explícitos y justificados.
5. Eliminar duplicados de forma controlada usando funciones de ventana.
6. Construir tablas limpias en una zona "clean" y una vista integrada de cliente en una zona "prepared".
7. Validar la calidad del resultado y documentar las decisiones de limpieza tomadas.
8. (Opcional) Utilizar AWS Glue DataBrew para perfilar y transformar datos sin código.

Conexión con el Tema 1 y el Tema 2

El Tema 1 nos enseñó que la traza es el lenguaje del sistema, pero también advirtió, en su sección 1.5, de los límites y sesgos de la traza. Este laboratorio es la materialización práctica de esa advertencia: no solo identificamos los problemas, sino que aprendemos a tratarlos. La calidad de datos no es una fase burocrática previa al "trabajo de verdad"; es parte sustancial

de la comprensión del sistema. Cada decisión de limpieza es, en el fondo, una hipótesis sobre cómo funciona realmente el sistema observado.

2. Contexto, servicios y datasets

Situación de partida

La dirección de la operadora ha aprobado un proyecto para "aplicar IA a la reducción del abandono y a la mejora de la calidad de servicio". El equipo de datos, tras el inventario del Lab 1, ha recibido cuatro extracciones de los sistemas operativos: el maestro de clientes, los tickets, las métricas de red y las reclamaciones. Al abrirlas, descubre que provienen de sistemas distintos, mantenidos por equipos distintos, con criterios distintos, y que no están listas para ningún análisis. Antes de modelar nada, hay que prepararlas.

Servicios AWS utilizados

Servicio	Función en el laboratorio
Amazon S3	Almacenar las zonas de datos: raw (crudo), clean (limpio) y prepared (integrado).
AWS Glue Data Catalog	Registrar el esquema de las tablas crudas y limpias.
Amazon Athena	Perfilar, limpiar e integrar los datos mediante consultas SQL y sentencias CTAS.
AWS Glue DataBrew (opcional)	Perfilar y transformar datos de forma visual, sin escribir código.

Tabla 1. Servicios AWS empleados en el Lab 2.

Introducimos aquí una idea arquitectónica muy extendida: la organización del data lake en zonas o capas. La zona raw contiene los datos tal como llegaron, sin tocar (es la fuente de verdad y nunca se modifica). La zona clean contiene versiones depuradas de cada dataset. La zona prepared contiene datos ya integrados y listos para alimentar modelos. Esta separación permite reproducir el proceso, auditar cada transformación y volver atrás si algo falla.

Datasets de partida

Se proporcionan cuatro ficheros CSV "sucios", que son versiones con problemas de calidad de los datos del caso telco:

Fichero	Filas aprox.	Problemas inyectados (a descubrir por el alumno)
customers_dirty.csv	520	Duplicados, nulos en precio/antigüedad, regiones y contratos inconsistentes, precios negativos, fechas futuras, "active" heterogéneo
tickets_dirty.csv	430	ticket_id duplicados, categorías y estados inconsistentes, espacios en blanco, claves de cliente huérfanas, horas de resolución negativas o absurdas
network_metrics_dirty.csv	1.038	Duplicados de (timestamp, región), valores negativos, outliers extremos, nulos, regiones inconsistentes

Fichero	Filas aprox.	Problemas inyectados (a descubrir por el alumno)
complaints_dirty.csv	128	complaint_id duplicados, etiquetas de sentimiento heterogéneas, textos vacíos, espacios en blanco, regiones inconsistentes

Tabla 2. Datasets sucios de partida.

Nota. Los problemas de calidad son intencionados y sintéticos, pero reproducen fielmente los que aparecen en extracciones reales de sistemas operativos. El número exacto de filas puede variar ligeramente por los duplicados aleatorios.

3. Preparación del entorno en AWS Academy

El procedimiento de inicio es idéntico al del Lab 1. Si ya lo realizaste, esta sección te resultará familiar.

Paso 3.1. Iniciar la sesión

1. Accede a AWS Academy desde el campus virtual y abre el Learner Lab de la asignatura.
2. Pulsa Start Lab y espera a que el círculo se ponga verde.
3. Pulsa AWS para abrir la consola y verifica que la región es us-east-1 (N. Virginia).

Paso 3.2. Crear el bucket y las zonas

Crea un bucket nuevo (o reutiliza el del Lab 1 si lo conservaste). Sugerencia de nombre: unir-lab2-telco-calidad-<iniciales>-<año>. Dentro, crea esta estructura de zonas:

```
raw/  
  customers/  
  tickets/  
  network_metrics/  
  complaints/  
clean/  
prepared/  
athena-results/
```

Buena práctica. La separación raw / clean / prepared es el corazón de este laboratorio. raw nunca se modifica; clean y prepared se generan a partir de raw mediante consultas. Si te equivocas, siempre puedes regenerar desde raw.

Paso 3.3. Subir los datasets crudos

Sube cada CSV sucio a su carpeta correspondiente dentro de raw/, igual que en el Lab 1:

Fichero	Carpeta destino
customers_dirty.csv	raw/customers/
tickets_dirty.csv	raw/tickets/
network_metrics_dirty.csv	raw/network_metrics/
complaints_dirty.csv	raw/complaints/

Tabla 3. Subida de datasets crudos.

Paso 3.4. Configurar Athena y crear la base de datos

En Athena, configura la ubicación de resultados en s3://<tu-bucket>/athena-results/ y crea la base de datos del laboratorio:

```
CREATE DATABASE IF NOT EXISTS unir_telco_lab2;
```

Selecciona unir_telco_lab2 en el desplegable Database antes de continuar.

Parte A. Registrar las tablas crudas

Registramos cada CSV sucio como tabla externa en la zona raw. Importante: definimos casi todas las columnas como string. Esto es deliberado: cuando los datos están sucios, forzar tipos numéricos en la carga provoca errores o pérdidas silenciosas de filas. Cargamos todo como texto y convertimos los tipos durante la limpieza, donde podemos controlar qué pasa con cada valor problemático.

Buena práctica. Cargar primero todo como string y tipar después es una práctica estándar en ingeniería de datos cuando la calidad del origen es desconocida. Evita que una sola celda corrupta tumbé la carga de un fichero entero.

Paso A.1. Tabla customers_raw

```
CREATE EXTERNAL TABLE IF NOT EXISTS customers_raw (  
    customer_id    string,  
    region         string,  
    contract_type  string,  
    monthly_price  string,  
    tenure_months  string,  
    signup_date    string,  
    active         string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ('separatorChar'=',', 'quoteChar'='\"', 'escapeChar'='\\')  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/customers/'  
TBLPROPERTIES ('skip.header.line.count'='1');
```

Paso A.2. Tabla tickets_raw

```
CREATE EXTERNAL TABLE IF NOT EXISTS tickets_raw (  
    ticket_id      string,  
    event_timestamp string,  
    customer_id    string,  
    category       string,  
    status         string,  
    priority       string,  
    resolution_hours string,  
    description     string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ('separatorChar'=',', 'quoteChar'='\"', 'escapeChar'='\\')  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/tickets/'  
TBLPROPERTIES ('skip.header.line.count'='1');
```

Paso A.3. Tabla network_metrics_raw

```
CREATE EXTERNAL TABLE IF NOT EXISTS network_metrics_raw (  
    event_timestamp string,  
    region          string,
```



```
    avg_download_mbps    string,  
    avg_upload_mbps      string,  
    avg_latency_ms       string,  
    packet_loss_pct      string,  
    active_users         string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ('separatorChar'=',', 'quoteChar'='"', 'escapeChar'='\\')  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/network_metrics/'  
TBLPROPERTIES ('skip.header.line.count'='1');
```

Paso A.4. Tabla complaints_raw

```
CREATE EXTERNAL TABLE IF NOT EXISTS complaints_raw (  
    complaint_id    string,  
    event_timestamp string,  
    customer_id     string,  
    region          string,  
    channel         string,  
    sentiment_label string,  
    text            string  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ('separatorChar'=',', 'quoteChar'='"', 'escapeChar'='\\')  
STORED AS TEXTFILE  
LOCATION 's3://<tu-bucket>/raw/complaints/'  
TBLPROPERTIES ('skip.header.line.count'='1');
```

Verifica las cargas con un recuento por tabla:

```
SELECT COUNT(*) FROM customers_raw;  
SELECT COUNT(*) FROM tickets_raw;  
SELECT COUNT(*) FROM network_metrics_raw;  
SELECT COUNT(*) FROM complaints_raw;
```

Parte B. Perfilado de datos

Antes de limpiar nada, hay que entender qué está mal. El perfilado (data profiling) consiste en describir estadísticamente cada columna para detectar anomalías. Es la fase diagnóstica de la calidad de datos.

Paso B.1. Nulos y vacíos por columna (customers)

OpenCSVSerde carga los valores ausentes como cadena vacía, no como NULL. Por eso comprobamos ambos casos:

```
SELECT
  SUM(CASE WHEN customer_id IS NULL OR customer_id = '' THEN 1 ELSE 0 END) AS
nulos_id,
  SUM(CASE WHEN region IS NULL OR region = '' THEN 1 ELSE 0 END) AS
nulos_region,
  SUM(CASE WHEN monthly_price IS NULL OR monthly_price = '' THEN 1 ELSE 0 END) AS
nulos_precio,
  SUM(CASE WHEN tenure_months IS NULL OR tenure_months = '' THEN 1 ELSE 0 END) AS
nulos_antiguedad,
  SUM(CASE WHEN active IS NULL OR active = '' THEN 1 ELSE 0 END) AS
nulos_active
FROM customers_raw;
```

Preguntas: ¿qué columnas tienen ausencias? ¿Son aleatorias o sistemáticas? ¿Qué columna es más crítica que falte?

Paso B.2. Detección de duplicados

Un dato maestro de cliente debería tener un customer_id único. Comprobémoslo:

```
SELECT customer_id, COUNT(*) AS veces
FROM customers_raw
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY veces DESC;
```

Repite el mismo patrón para tickets_raw (sobre ticket_id) y complaints_raw (sobre complaint_id). Para network_metrics, la clave natural es la combinación de timestamp y región:

```
SELECT event_timestamp, region, COUNT(*) AS veces
FROM network_metrics_raw
GROUP BY event_timestamp, region
HAVING COUNT(*) > 1
ORDER BY veces DESC
LIMIT 20;
```

Paso B.3. Cardinalidad y categorías inconsistentes

Una de las patologías más frecuentes es la misma categoría escrita de formas distintas. Veámoslo en region y contract_type:

```
SELECT region, COUNT(*) AS total
FROM customers_raw
GROUP BY region
ORDER BY region;
```

```
SELECT contract_type, COUNT(*) AS total
FROM customers_raw
GROUP BY contract_type
ORDER BY contract_type;
```

Verás que "Madrid", "MADRID", "madrid" y " Madrid " aparecen como categorías diferentes, cuando son la misma. Lo mismo con los tipos de contrato. Esto rompe cualquier agregación por región o por producto.

Atención. Si agrupas por una categoría sucia, los totales se reparten entre las variantes y todas las cifras quedan mal. Es uno de los errores más caros y más invisibles del análisis de datos.

Paso B.4. Rangos y valores imposibles

Para las columnas numéricas (que están como texto), convertimos al vuelo y miramos los extremos. En precio:

```
SELECT
  MIN(TRY_CAST(monthly_price AS double)) AS precio_min,
  MAX(TRY_CAST(monthly_price AS double)) AS precio_max,
  AVG(TRY_CAST(monthly_price AS double)) AS precio_medio
FROM customers_raw;
```

Un precio mínimo negativo o un máximo desorbitado son señales de valores imposibles. TRY_CAST devuelve NULL cuando no puede convertir, en lugar de fallar, lo que lo hace ideal para datos sucios. Repite el análisis en network_metrics para download y latencia:

```
SELECT
  MIN(TRY_CAST(avg_download_mbps AS double)) AS dl_min,
  MAX(TRY_CAST(avg_download_mbps AS double)) AS dl_max,
  MIN(TRY_CAST(avg_latency_ms AS double)) AS lat_min,
  MAX(TRY_CAST(avg_latency_ms AS double)) AS lat_max
FROM network_metrics_raw;
```

Paso B.5. Fechas imposibles

Un alta de cliente no puede tener fecha futura. Detectemos las fechas posteriores a la extracción (noviembre de 2025):

```
SELECT customer_id, signup_date
FROM customers_raw
WHERE signup_date > '2025-11-01'
ORDER BY signup_date DESC;
```

Paso B.6. Integridad referencial (claves huérfanas)

Los tickets deben pertenecer a clientes existentes. Busquemos tickets cuyo customer_id no esté en el maestro:

```
SELECT t.customer_id, COUNT(*) AS tickets_huerfanos
FROM tickets_raw t
LEFT JOIN customers_raw c ON t.customer_id = c.customer_id
WHERE c.customer_id IS NULL
GROUP BY t.customer_id
ORDER BY tickets_huerfanos DESC;
```

Preguntas: ¿cuántos tickets tienen cliente inexistente? ¿Los descartamos, los marcamos o investigamos su origen? No hay una única respuesta correcta; lo importante es decidir con criterio y documentarlo.

Parte C. Limpieza y estandarización

Con el diagnóstico hecho, pasamos a corregir. Cada corrección será una transformación SQL explícita. Al final, materializaremos los resultados en la zona clean con sentencias CTAS (CREATE TABLE AS SELECT).

Paso C.1. Estandarizar texto: regiones y contratos

Para normalizar categorías de texto aplicamos tres funciones combinadas: TRIM (quita espacios), UPPER o LOWER (unifica mayúsculas) y, cuando hace falta, REPLACE (corrige variantes). Probemos primero la normalización de región:

```
SELECT DISTINCT
  region                                AS region_original,
  UPPER(TRIM(region))                  AS region_normalizada
FROM customers_raw
ORDER BY region_normalizada;
```

Para los contratos, además de TRIM y minúsculas, hay que recomponer el guion bajo que faltaba en variantes como "movilsolo" o "fibra600". Una forma robusta es mapear explícitamente:

```
SELECT DISTINCT contract_type AS original,
  CASE
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra600', 'fibra_600')
  THEN 'fibra_600'
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra300', 'fibra_300')
  THEN 'fibra_300'
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra1000', 'fibra_1000')
  THEN 'fibra_1000'
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('movilsolo', 'movil_solo')
  THEN 'movil_solo'
    WHEN lower(replace(trim(contract_type), ' ', '')) = 'convergente' THEN
'convergente'
    ELSE 'desconocido'
  END AS contract_normalizado
FROM customers_raw
ORDER BY original;
```

Paso C.2. Normalizar el campo "active"

El campo active mezcla 1/0, yes/no y vacíos. Lo unificamos a un entero 0/1, dejando NULL lo que no se pueda determinar:

```
SELECT DISTINCT active AS original,
  CASE
    WHEN lower(trim(active)) IN ('1', 'yes', 'si', 'true') THEN 1
    WHEN lower(trim(active)) IN ('0', 'no', 'false')      THEN 0
    ELSE NULL
  END AS active_norm
FROM customers_raw;
```

Paso C.3. Eliminar duplicados con función de ventana

Para quedarnos con una sola fila por cliente usamos ROW_NUMBER(), numerando las filas de cada customer_id y conservando solo la primera. Como criterio de desempate, preferimos la fila con menos campos vacíos (aquí, simplificando, ordenamos por signup_date):

```
WITH numeradas AS (  
  SELECT *,  
    ROW_NUMBER() OVER (  
      PARTITION BY customer_id  
      ORDER BY signup_date  
    ) AS rn  
  FROM customers_raw  
)  
SELECT COUNT(*) AS filas_unicas  
FROM numeradas  
WHERE rn = 1;
```

El resultado debería acercarse a 500 (los clientes únicos), frente a las 520 filas originales.

Paso C.4. Tratar valores imposibles

Definimos reglas explícitas: un precio negativo o nulo se marca como NULL (no inventamos un valor); una antigüedad vacía queda como NULL; una fecha de alta futura se marca como NULL. La política de "marcar como desconocido en lugar de inventar" es, en general, más honesta que la imputación silenciosa.

Buena práctica. Imputar (rellenar) valores ausentes con la media o la moda puede ser legítimo, pero debe ser una decisión consciente y documentada, no un automatismo. En este lab optamos por marcar como NULL y dejar que el modelo o el analista decida después.

Paso C.5. Construir la tabla limpia de clientes (CTAS)

Reunimos todas las transformaciones en una sola sentencia que crea la tabla limpia y escribe el resultado en la zona clean de S3:

```
CREATE TABLE customers_clean  
WITH (  
  external_location = 's3://<tu-bucket>/clean/customers_clean/',  
  format = 'PARQUET'  
) AS  
WITH numeradas AS (  
  SELECT *,  
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY signup_date) AS rn  
  FROM customers_raw  
)  
SELECT  
  customer_id,  
  UPPER(TRIM(region)) AS region,  
  CASE  
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra600', 'fibra_600')  
  THEN 'fibra_600'  
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra300', 'fibra_300')  
  THEN 'fibra_300'
```

```

    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('fibra1000', 'fibra_1000')
THEN 'fibra_1000'
    WHEN lower(replace(trim(contract_type), ' ', '')) IN ('movilsolo', 'movil_solo')
THEN 'movil_solo'
    WHEN lower(replace(trim(contract_type), ' ', '')) = 'convergente'
THEN 'convergente'
    ELSE 'desconocido'
END AS contract_type,
CASE WHEN TRY_CAST(monthly_price AS double) > 0
      THEN TRY_CAST(monthly_price AS double) END AS monthly_price,
TRY_CAST(tenure_months AS int) AS tenure_months,
CASE WHEN signup_date <= '2025-11-01' THEN signup_date END AS signup_date,
CASE
  WHEN lower(trim(active)) IN ('1', 'yes', 'si', 'true') THEN 1
  WHEN lower(trim(active)) IN ('0', 'no', 'false')      THEN 0
END AS active
FROM numeradas
WHERE rn = 1;

```

Buena práctica. Guardamos en formato PARQUET, columnar y comprimido, que es mucho más eficiente que CSV para consultas analíticas. Es el formato recomendado para las zonas clean y prepared.

Paso C.6. Limpiar tickets

Aplicamos el mismo enfoque a los tickets: deduplicar por ticket_id, estandarizar categoría y estado, recortar el texto y tratar las horas de resolución imposibles (negativas o absurdas se marcan como NULL):

```

CREATE TABLE tickets_clean
WITH (
  external_location = 's3://<tu-bucket>/clean/tickets_clean/',
  format = 'PARQUET'
) AS
WITH numeradas AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY ticket_id ORDER BY event_timestamp) AS rn
  FROM tickets_raw
)
SELECT
  ticket_id,
  event_timestamp,
  customer_id,
  lower(replace(trim(category), ' ', '_')) AS category,
  lower(trim(status))                      AS status,
  lower(trim(priority))                    AS priority,
  CASE WHEN TRY_CAST(resolution_hours AS double) BETWEEN 0 AND 720
        THEN TRY_CAST(resolution_hours AS double) END AS resolution_hours,
  TRIM(description) AS description
FROM numeradas
WHERE rn = 1;

```

Paso C.7. Limpiar métricas de red

```

CREATE TABLE network_metrics_clean
WITH (
  external_location = 's3://<tu-bucket>/clean/network_metrics_clean/',
  format = 'PARQUET'
) AS
WITH numeradas AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY event_timestamp, UPPER(TRIM(region))
      ORDER BY event_timestamp
    ) AS rn
  FROM network_metrics_raw
)
SELECT
  event_timestamp,
  UPPER(TRIM(region)) AS region,
  CASE WHEN TRY_CAST(avg_download_mbps AS double) BETWEEN 0 AND 2000
    THEN TRY_CAST(avg_download_mbps AS double) END AS avg_download_mbps,
  TRY_CAST(avg_upload_mbps AS double) AS avg_upload_mbps,
  CASE WHEN TRY_CAST(avg_latency_ms AS double) BETWEEN 0 AND 500
    THEN TRY_CAST(avg_latency_ms AS double) END AS avg_latency_ms,
  CASE WHEN TRY_CAST(packet_loss_pct AS double) BETWEEN 0 AND 5
    THEN TRY_CAST(packet_loss_pct AS double) END AS packet_loss_pct,
  TRY_CAST(active_users AS int) AS active_users
FROM numeradas
WHERE rn = 1;

```

Paso C.8. Limpiar reclamaciones

```

CREATE TABLE complaints_clean
WITH (
  external_location = 's3://<tu-bucket>/clean/complaints_clean/',
  format = 'PARQUET'
) AS
WITH numeradas AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY complaint_id ORDER BY event_timestamp) AS rn
  FROM complaints_raw
)
SELECT
  complaint_id,
  event_timestamp,
  customer_id,
  UPPER(TRIM(region)) AS region,
  lower(trim(channel)) AS channel,
  CASE
    WHEN lower(trim(sentiment_label)) IN ('negativo','neg') THEN 'negativo'
    WHEN lower(trim(sentiment_label)) IN ('neutral','neu') THEN 'neutral'
    WHEN lower(trim(sentiment_label)) IN ('positivo','pos') THEN 'positivo'
    ELSE 'desconocido'
  END AS sentiment_label,
  TRIM(text) AS text
FROM numeradas

```



```
WHERE rn = 1 AND TRIM(text) <> '';
```

Parte D. Integración: la vista de cliente

Datos limpios pero aislados sirven de poco. El valor analítico aparece cuando integramos las distintas trazas en torno a una entidad común: el cliente. Construiremos una tabla integrada (a veces llamada "customer 360" o tabla maestra de features) que reúne, por cada cliente, sus datos de contrato y un resumen de su actividad en tickets y reclamaciones.

Paso D.1. Agregados de tickets por cliente

```
SELECT
  customer_id,
  COUNT(*) AS num_tickets,
  SUM(CASE WHEN priority IN ('alta','critica') THEN 1 ELSE 0 END) AS
tickets_prioritarios,
  AVG(resolution_hours) AS resolution_media_horas
FROM tickets_clean
GROUP BY customer_id;
```

Paso D.2. Agregados de reclamaciones por cliente

```
SELECT
  customer_id,
  COUNT(*) AS num_reclamaciones,
  SUM(CASE WHEN sentiment_label = 'negativo' THEN 1 ELSE 0 END) AS
reclamaciones_negativas
FROM complaints_clean
GROUP BY customer_id;
```

Paso D.3. Construir la tabla integrada (zona prepared)

Unimos el maestro limpio con los agregados mediante LEFT JOIN, de modo que conservemos a todos los clientes aunque no tengan tickets ni reclamaciones. Donde no haya actividad, COALESCE pone 0:

```
CREATE TABLE customer_360
WITH (
  external_location = 's3://<tu-bucket>/prepared/customer_360/',
  format = 'PARQUET'
) AS
WITH t AS (
  SELECT customer_id,
    COUNT(*) AS num_tickets,
    SUM(CASE WHEN priority IN ('alta','critica') THEN 1 ELSE 0 END) AS
tickets_prioritarios,
    AVG(resolution_hours) AS resolution_media_horas
  FROM tickets_clean GROUP BY customer_id
),
r AS (
  SELECT customer_id,
    COUNT(*) AS num_reclamaciones,
    SUM(CASE WHEN sentiment_label = 'negativo' THEN 1 ELSE 0 END) AS
reclamaciones_negativas
```

```
FROM complaints_clean GROUP BY customer_id
)
SELECT
  c.customer_id,
  c.region,
  c.contract_type,
  c.monthly_price,
  c.tenure_months,
  c.active,
  COALESCE(t.num_tickets, 0) AS num_tickets,
  COALESCE(t.tickets_prioritarios, 0) AS tickets_prioritarios,
  t.resolucion_media_horas,
  COALESCE(r.num_reclamaciones, 0) AS num_reclamaciones,
  COALESCE(r.reclamaciones_negativas, 0) AS reclamaciones_negativas
FROM customers_clean c
LEFT JOIN t ON c.customer_id = t.customer_id
LEFT JOIN r ON c.customer_id = r.customer_id;
```

Esta tabla `customer_360` es exactamente el tipo de insumo que los Labs 3 y 4 necesitarán para formular y entrenar modelos. Acabamos de construir, de forma trazable y reproducible, el puente entre la traza cruda y la inteligencia artificial.

Paso D.4. Inspeccionar el resultado

```
SELECT * FROM customer_360
ORDER BY num_tickets DESC
LIMIT 20;
```

Parte E. Validación de la calidad

La limpieza no termina al crear las tablas: hay que validar que efectivamente quedaron limpias. Esta fase de control es lo que distingue un proceso profesional de uno improvisado.

Paso E.1. Comprobar unicidad

```
SELECT COUNT(*) AS filas, COUNT(DISTINCT customer_id) AS clientes_unicos
FROM customers_clean;
```

Si ambos números coinciden, ya no hay duplicados de cliente.

Paso E.2. Comprobar categorías ya normalizadas

```
SELECT region, COUNT(*) FROM customers_clean GROUP BY region ORDER BY region;
SELECT contract_type, COUNT(*) FROM customers_clean GROUP BY contract_type ORDER
BY contract_type;
```

Deberías ver exactamente seis regiones y cinco tipos de contrato (más, quizá, "desconocido"), no decenas de variantes.

Paso E.3. Comprobar que ya no hay valores imposibles

```
SELECT MIN(monthly_price) AS precio_min, MAX(monthly_price) AS precio_max
FROM customers_clean;
SELECT MIN(avg_download_mbps) AS dl_min, MAX(avg_download_mbps) AS dl_max
FROM network_metrics_clean;
```

Paso E.4. Cuadro de mando de calidad

Como cierre, conviene producir un pequeño resumen comparativo de antes y después. Completa una tabla como esta a partir de tus consultas de las partes B y E:

Indicador	Antes (raw)	Después (clean)
Filas de clientes	520	≈ 500
Variantes de región	≈ 29	6
Variantes de contrato	≈ 19	5 (+ desconocido)
Precios negativos / nulos	> 30	0 (marcados NULL)
ticket_id duplicados	10	0
(timestamp, región) duplicados	30	0
Sentimientos heterogéneos	> 8 variantes	3 (+ desconocido)

Tabla 4. Cuadro de mando de calidad: antes y después.

Parte F. (Opcional) AWS Glue DataBrew

Todo lo anterior se ha hecho con SQL en Athena, que es el enfoque más transparente y reproducible. Como alternativa visual y sin código, AWS Glue DataBrew permite perfilar y transformar datos mediante una interfaz gráfica. Esta parte es opcional y puede no estar disponible en todas las configuraciones de AWS Academy.

Paso F.1. Crear un dataset en DataBrew

1. En la consola, busca DataBrew y ábrelo.
2. Pulsa Create dataset, dale un nombre (por ejemplo customers_dirty) y selecciona como origen el CSV en s3://<tu-bucket>/raw/customers/.
3. Confirma la creación del dataset.

Paso F.2. Perfilar

4. Selecciona el dataset y pulsa Run data profile.
5. Indica como salida una ruta en s3://<tu-bucket>/athena-results/profile/ y lanza el trabajo.
6. Cuando termine, revisa el informe: muestra porcentaje de nulos, valores distintos, distribución y correlaciones por columna, de forma visual.

Paso F.3. Crear un proyecto de transformación

7. Crea un Project sobre el dataset.
8. En el editor visual, aplica recetas (recipes): eliminar duplicados, normalizar mayúsculas, recortar espacios, filtrar valores fuera de rango.
9. Cada paso queda registrado en la receta, que es reproducible y exportable.

Nota. DataBrew es muy útil para perfilar rápido y para perfiles no técnicos, pero el control fino y la reproducibilidad plena se consiguen mejor con SQL. Compara ambos enfoques y reflexiona sobre cuándo conviene cada uno.

Atención. DataBrew genera costes por sesión de proyecto y por trabajo de perfilado. En AWS Academy, úsalo con moderación y elimina proyectos y trabajos al terminar.

Parte G. Entregable del alumno

El alumno entregará un documento de entre cinco y siete páginas con los siguientes apartados:

1. Informe de perfilado. Para cada dataset, los principales problemas de calidad detectados, con las consultas usadas y una captura del resultado.
2. Registro de decisiones de limpieza. Tabla en la que, para cada problema, se indique: qué se hizo (eliminar, normalizar, marcar NULL, filtrar), por qué, y qué alternativa se descartó.
3. Sentencias CTAS finales. El SQL de creación de las cuatro tablas clean y de la tabla customer_360.
4. Cuadro de mando de calidad antes/después. La Tabla 4 completada con tus cifras reales.
5. Inspección de customer_360. Captura de las primeras filas y comentario sobre su utilidad para los próximos labs.
6. Reflexión crítica. Media página sobre el siguiente dilema: toda limpieza es una interpretación. ¿Qué riesgos introduce limpiar? ¿Podríamos estar borrando señal real al eliminar "outliers"? ¿Qué decisiones de limpieza podrían sesgar un modelo posterior?

Criterios de evaluación

Criterio	Peso	Insuficiente	Aceptable	Excelente
Perfilado	20%	No detecta problemas	Detecta los principales	Perfilado sistemático y cuantificado
Limpieza y estandarización	25%	Transformaciones erróneas	Tablas clean correctas	Transformaciones robustas y justificadas
Integración (customer_360)	20%	No integra o pierde clientes	Integra correctamente	LEFT JOIN correcto, COALESCE, sin pérdidas
Validación	15%	No valida	Valida lo básico	Cuadro antes/después completo
Registro de decisiones	10%	Ausente	Presente	Justifica y discute alternativas
Reflexión crítica	10%	Trivial	Adecuada	Distingue limpiar de sesgar, con ejemplos

Tabla 5. Criterios y rúbrica de evaluación.

Parte H. Limpieza de recursos y cierre

Paso H.1. Eliminar tablas y base de datos

```
DROP TABLE IF EXISTS customers_raw;  
DROP TABLE IF EXISTS tickets_raw;  
DROP TABLE IF EXISTS network_metrics_raw;  
DROP TABLE IF EXISTS complaints_raw;  
DROP TABLE IF EXISTS customers_clean;  
DROP TABLE IF EXISTS tickets_clean;  
DROP TABLE IF EXISTS network_metrics_clean;  
DROP TABLE IF EXISTS complaints_clean;  
DROP TABLE IF EXISTS customer_360;  
DROP DATABASE IF EXISTS unir_telco_lab2;
```

Atención. Al borrar tablas creadas con CTAS, Athena NO elimina los datos PARQUET en S3 (solo el catálogo). Si quieres liberar el almacenamiento por completo, vacía también las carpetas clean/ y prepared/ del bucket.

Paso H.2. Vaciar y eliminar el bucket

Si no vas a reutilizar los datos en el Lab 3, vacía el bucket (Empty) y elimínalo (Delete) desde la consola de S3. Si sí piensas continuar, conserva al menos la zona prepared con la tabla customer_360.

Paso H.3. End Lab

1. Vuelve a la pestaña del Learner Lab y pulsa End Lab.
2. Confirma y verifica que el círculo pasa a rojo.

Cierre didáctico

Has transformado cuatro extracciones sucias e inutilizables en una capa de datos limpia, integrada y lista para la inteligencia artificial. Por el camino has aprendido a perfilar, a estandarizar, a deduplicar, a tratar valores imposibles y a integrar trazas heterogéneas en torno al cliente. Más importante aún: has comprendido que la limpieza no es un trámite, sino una secuencia de decisiones interpretativas, cada una con consecuencias para los modelos que vendrán después.

La tabla customer_360 que has construido será el punto de partida natural del Lab 3, donde formularemos casos de IA, y del Lab 4, donde entrenaremos un modelo de churn. Llegamos a esos labs no con datos crudos y dudosos, sino con una base depurada, trazable y auditable. Esa es exactamente la diferencia entre la analítica artesanal y la ingeniería de datos profesional.

Referencias y lecturas complementarias

- Amazon Web Services. (2024). Amazon Athena User Guide — CTAS and data types. <https://docs.aws.amazon.com/athena/>

- Amazon Web Services. (2024). AWS Glue DataBrew Developer Guide. <https://docs.aws.amazon.com/databrew/>
- Amazon Web Services. (2024). Best practices for organizing a data lake on Amazon S3. <https://docs.aws.amazon.com/>
- Kim, W., Choi, B. J., Hong, E. K., Kim, S. K., & Lee, D. (2003). A taxonomy of dirty data. *Data Mining and Knowledge Discovery*, 7(1), 81-99.
- Kitchin, R. (2014). *The data revolution: Big data, open data, data infrastructures and their consequences*. SAGE Publications.
- Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin*, 23(4), 3-13.